Авторские права
© Postgres Professional, 2017–2024
Авторы: Егор Рогов, Павел Лузанов, Илья Баштанов, Алексей Береснев
Фото: Олег Бартунов (монастырь Пху и пик Бхрикути, Непал)

Использование материалов курса
Некоммерческое использование материалов курса (презентации, демонстрации) разрешается без ограничений. Коммерческое использование возможно только с письменного разрешения компании Postgres Professional. Запрещается внесение изменений в материалы курса.

Обратная связь
Отзывы, замечания и предложения направляйте по адресу:
edu@postgrespro.ru

Отказ от ответственности
Компания Postgres Professional не несет никакой ответственности за любые повреждения и убытки, включая потерю дохода, нанесенные прямым или непрямым, специальным или случайным использованием материалов курса. Компания Postgres Professional не предоставляет каких-либо гарантий на материалы курса. Материалы курса предоставляются на основе принципа «как есть» и компания Postgres Professional не обязана предоставлять сопровождение, поддержку, обновления, расширения и изменения.

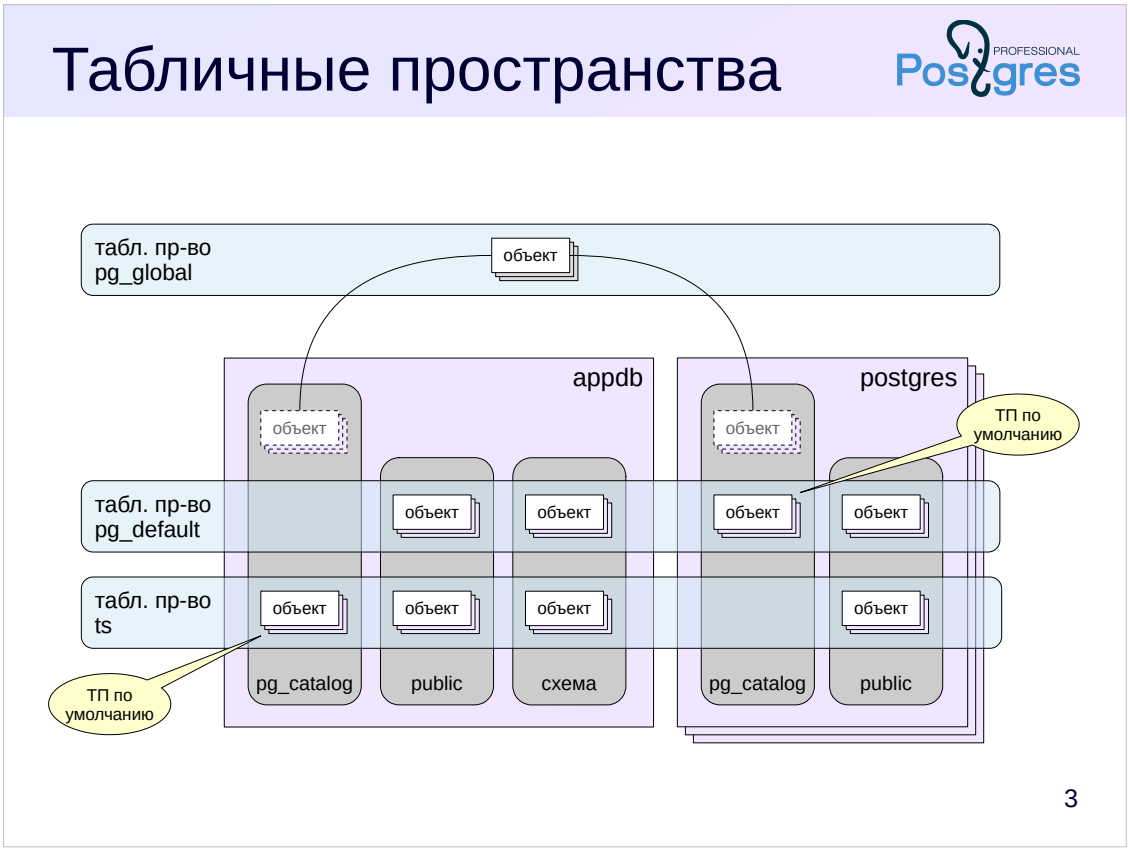
Темы

Табличные пространства и каталоги

Создание, изменение и удаление табличных пространств

Хранение данных в файловой системе

Перемещение данных



Табличные пространства (ТП) служат для организации физического хранения данных и определяют расположение данных в файловой системе.

Например, можно создать одно ТП на медленных дисках для архивных данных, а другое — на быстрых дисках для данных, с которыми идет активная работа.

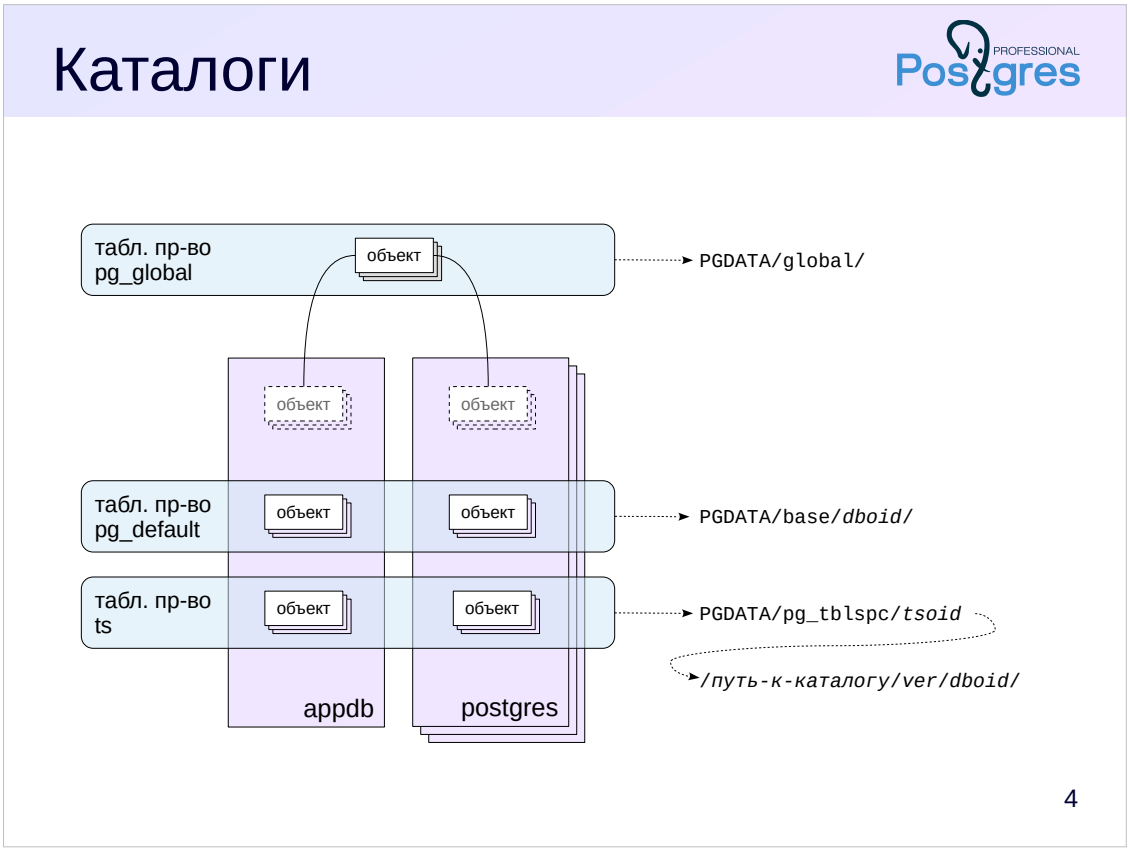
При инициализации кластера создаются два ТП: pg_default и pg_global.

Одно и то же ТП может использоваться разными базами данных, а одна база данных может хранить данные в нескольких ТП.

При этом у каждой БД есть так называемое «ТП по умолчанию», в котором создаются все объекты, если не указано иное. В этом же ТП хранятся и объекты системного каталога. Изначально в качестве «ТП по умолчанию» используется ТП pg_default, но можно установить и другое.

ТП pg_global особенное: в нем хранятся те объекты системного каталога, которые являются общими для кластера.

https://postgrespro.ru/docs/postgresql/16/manage-ag-tablespaces



По сути, табличное пространство — это указание на каталог, в котором располагаются данные. Стандартные ТП pg_global и pg_default всегда находятся в PGDATA/global/ и PGDATA/base/ соответственно. При создании пользовательского ТП указывается произвольный каталог; для собственного удобства PostgreSQL создает на него символическую ссылку в каталоге PGDATA/pg_tblspc/.

Внутри каталога PGDATA/base/ данные дополнительно разложены по подкаталогам баз данных (для PGDATA/global/ это не требуется, так как данные в нем относятся к кластеру в целом).

Внутри каталога пользовательского ТП появляется еще один уровень вложенности: версия сервера PostgreSQL. Это сделано для удобства обновления сервера на другую версию.

Собственно объекты хранятся в файлах внутри этих каталогов — каждый объект в отдельных файлах.

Системные табличные пространства

При создании кластера создаются два табличных пространства:

```
=> SELECT * FROM pg_tablespace;
```

```
 oid  |  spcname   | spcowner | spcacl | spcoptions
------+------------+----------+--------+------------
 1663 | pg_default |       10 |        |
 1664 | pg_global  |       10 |        |
(2 rows)
```

- pg_global — общие объекты кластера;
- pg_default — табличное пространство по умолчанию.

Пользовательские табличные пространства

Для нового табличного пространства нужен пустой каталог, владельцем которого является пользователь postgres.

```
student$ sudo -u postgres mkdir /var/lib/postgresql/ts_dir
```

Теперь можно создать табличное пространство:

```
=> CREATE TABLESPACE ts LOCATION '/var/lib/postgresql/ts_dir';
```

```
CREATE TABLESPACE
```

Список табличных пространств можно получить и командой psql:

```
=> \db
```

```
                List of tablespaces
    Name    |  Owner   |          Location
------------+----------+---------------------------
 pg_default | postgres |
 pg_global  | postgres |
 ts         | student  | /var/lib/postgresql/ts_dir
(3 rows)
```

У каждой базы данных есть табличное пространство «по умолчанию». Создадим базу и назначим ей ts в качестве такого пространства:

```
=> CREATE DATABASE appdb TABLESPACE ts;
```

```
CREATE DATABASE
```

Теперь все создаваемые таблицы и индексы будут попадать в ts, если явно не указать другое.

Подключимся к базе:

```
=> \c appdb
```

```
You are now connected to database "appdb" as user "student".
```

Создадим таблицу:

```
=> CREATE TABLE t1(
  id integer GENERATED ALWAYS AS IDENTITY,
  name text
);
```

```
CREATE TABLE
```

При создании объекта можно указать табличное пространство явно:

```
=> CREATE TABLE t2(
  n numeric
) TABLESPACE pg_default;
```

```
CREATE TABLE
```

```
=> SELECT tablename, tablespace FROM pg_tables WHERE schemaname = 'public';
```

```
 tablename | tablespace
-----------+------------
 t1        |
 t2        | pg_default
(2 rows)
```

Пустое поле tablespace указывает на табличное пространство по умолчанию, а у второй таблицы поле заполнено.

Для индекса также можно указать табличное пространство:

```
=> CREATE INDEX ON t1(id) TABLESPACE pg_default;
```

```
CREATE INDEX
```

```
=> SELECT indexname,tablespace FROM pg_indexes WHERE tablename='t1';
```

```
 indexname | tablespace
-----------+------------
 t1_id_idx | pg_default
(1 row)
```

Еще один способ задать табличное пространство без явного указания при создании объекта — предварительно установить значение параметра default_tablespace.

Одно табличное пространство может использоваться для объектов нескольких баз данных.

```
=> CREATE DATABASE configdb;
```

```
CREATE DATABASE
```

У этой БД табличным пространством по умолчанию будет pg_default.

```
=> \c configdb
```

```
You are now connected to database "configdb" as user "student".
```

```
=> CREATE TABLE t(
  n integer
) TABLESPACE ts;
```

```
CREATE TABLE
```

```
=> \d t
```

```
                 Table "public.t"
 Column |  Type   | Collation | Nullable | Default
--------+---------+-----------+----------+---------
 n      | integer |           |          |
Tablespace: "ts"
```

Для временных таблиц и их индексов можно указать отдельное табличное пространство по умолчанию:

```
=> SET temp_tablespaces = 'ts';
```

```
SET
```

```
=> CREATE TEMP TABLE temp(s text);
```

```
CREATE TABLE
```

```
=> \d temp
```

```
             Table "pg_temp_3.temp"
 Column | Type | Collation | Nullable | Default
--------+------+-----------+----------+---------
 s      | text |           |          |
Tablespace: "ts"
```

В параметре temp_tablespaces можно указать несколько табличных пространств, в этом случае сервер выберет одно из них случайным образом.

Управление объектами в табличных пространствах

Таблицы (и другие объекты, например, индексы), можно перемещать между табличными пространствами.

```
=> \c appdb
```

```
You are now connected to database "appdb" as user "student".
```

```
=> ALTER TABLE t1 SET TABLESPACE pg_default;
```

```
ALTER TABLE
```

```
=> SELECT tablename, tablespace FROM pg_tables WHERE schemaname = 'public';
```

```
 tablename | tablespace
-----------+------------
 t2        | pg_default
 t1        | pg_default
(2 rows)
```

Перенести индексы можно и при перестроении:

```
=> REINDEX (TABLESPACE ts) TABLE t1;
```

```
REINDEX
```

Можно переместить и все объекты из одного табличного пространства в другое:

```
=> ALTER TABLE ALL IN TABLESPACE pg_default SET TABLESPACE ts;
```

```
ALTER TABLE
```

```
=> SELECT tablename, tablespace FROM pg_tables WHERE schemaname = 'public';
```

```
 tablename | tablespace
-----------+------------
 t2        |
 t1        |
(2 rows)
```

Важно понимать, что перенос в другое табличное пространство (в отличие от переноса в другую схему) — физическая операция, связанная с копированием файлов данных из каталога в каталог. На время ее выполнения доступ к перемещаемому объекту полностью блокируется.

Размер табличного пространства

Мы уже рассматривали, как узнать объем, занимаемый базой данных. Можно узнать и объем объектов в табличном пространстве:

```
=> SELECT pg_size_pretty( pg_tablespace_size('ts') );
```

```
 pg_size_pretty
----------------
 7588 kB
(1 row)
```

Почему размер так велик, хотя в табличном пространстве всего несколько пустых таблиц?

Поскольку ts является табличным пространством по умолчанию для базы appdb, в нем хранятся объекты системного каталога. Они и занимают место.

В psql получить размер табличных пространств можно командой:

```
=> \db+
```

```
                                           List of tablespaces
    Name    |  Owner   |          Location          | Access privileges | Options |  Size
  | Description
------------+----------+----------------------------+-------------------+---------+-------
--+-------------
 pg_default | postgres |                            |                   |         | 36 MB
  |
 pg_global  | postgres |                            |                   |         | 589
kB  |
 ts         | student  | /var/lib/postgresql/ts_dir |                   |         | 7588
kB |
(3 rows)
```

Удаление табличного пространства

Табличное пространство можно удалить, но только в том случае, если оно пусто:

```
=> DROP TABLESPACE ts;
```

```
ERROR:  tablespace "ts" is not empty
```

В отличие от удаления схемы, в команде DROP TABLESPACE нельзя использовать предложение CASCADE: объекты табличного пространства могут принадлежать разным базам данных, а подключены мы только к одной.

Но можно выяснить, в каких базах есть зависимые объекты. В этом нам поможет системный каталог.

Сначала узнаем и запомним идентификатор табличного пространства:

```
=> SELECT oid FROM pg_tablespace WHERE spcname = 'ts';
```

```
  oid
-------
 16386
(1 row)
```

Затем получим список баз данных, в которых есть объекты из удаляемого пространства:

```
=> SELECT datname
FROM pg_database
WHERE oid IN (SELECT pg_tablespace_databases(16386));
```

```
 datname
----------
 configdb
 appdb
(2 rows)
```

Дальше подключаемся к каждой базе данных и получаем список объектов из pg_class:

```
=> \c configdb
```

```
You are now connected to database "configdb" as user "student".
```

```
=> SELECT relnamespace::regnamespace, relname, relkind
FROM pg_class
WHERE reltablespace = 16386;
```

```
 relnamespace | relname | relkind
--------------+---------+---------
 public       | t       | r
(1 row)
```

Таблица больше не нужна, удалим ее.

```
=> DROP TABLE t;
```

```
DROP TABLE
```

И вторая база данных. Поскольку ts является табличным пространством по умолчанию, у объектов в pg_class идентификатор табличного пространства равен нулю. Это, как нам уже известно, объекты системного каталога:

```
=> \c appdb
```

```
You are now connected to database "appdb" as user "student".
```

```
=> SELECT count(*) FROM pg_class WHERE reltablespace = 0;
```

```
 count
-------
   371
(1 row)
```

Табличное пространство по умолчанию можно сменить; при этом все таблицы из старого пространства физически переносятся в новое. Предварительно надо отключиться от базы.

```
=> \c postgres
```

```
You are now connected to database "postgres" as user "student".
```

```
=> ALTER DATABASE appdb SET TABLESPACE pg_default;
```

```
ALTER DATABASE
```

Вот теперь табличное пространство может быть удалено.

```
=> DROP TABLESPACE ts;
```

```
DROP TABLESPACE
```

От имени пользователя postgres удалим каталог, созданный для табличного пространства.

```
student$ sudo -u postgres rm -rf /var/lib/postgresql/ts_dir
```

Итоги

Табличные пространства — средство для организации физического хранения данных

Логическое (базы данных, схемы) и *физическое* (табличные пространства) разделения данных независимы

Практика

Почему при создании базы данных без предложения TABLESPACE табличным пространством по умолчанию становится pg_default?

1. Создайте новое табличное пространство.
2. Измените табличное пространство по умолчанию для базы данных template1 на созданное пространство.
3. Создайте новую базу данных.
   Проверьте, какое табличное пространство по умолчанию установлено для новой базы данных.
4. Найдите в PGDATA символическую ссылку на каталог табличного пространства.
5. Удалите созданное табличное пространство.

1. Новое табличное пространство

Каждая база данных имеет табличное пространство по умолчанию. В том числе и template1:

```
=> \x \l+ template1 \x
```

```
Expanded display is on.
List of databases
-[ RECORD 1 ]-----+-----------------------------------
Name              | template1
Owner             | postgres
Encoding          | UTF8
Locale Provider   | libc
Collate           | en_US.UTF-8
Ctype             | en_US.UTF-8
ICU Locale        |
ICU Rules         |
Access privileges | =c/postgres                       +
                  | postgres=CTc/postgres
Size              | 7516 kB
Tablespace        | pg_default
Description       | default template for new databases

Expanded display is off.
```

По умолчанию новые базы данных создаются копированием template1, поэтому они получают такое же табличное пространство, как установлено для template1.

Создадим новое табличное пространство.

```
student$ sudo -u postgres mkdir /var/lib/postgresql/ts_dir
```

```
=> CREATE TABLESPACE ts LOCATION '/var/lib/postgresql/ts_dir';
```

```
CREATE TABLESPACE
```

2. Табличное пространство по умолчанию для template1

```
=> ALTER DATABASE template1 SET TABLESPACE ts;
```

```
ALTER DATABASE
```

3. Новая база данных и проверка

```
=> CREATE DATABASE db;
```

```
CREATE DATABASE
```

```
=> SELECT spcname
FROM pg_tablespace
WHERE oid = (SELECT dattablespace FROM pg_database WHERE datname = 'db');
```

```
 spcname
---------
 ts
(1 row)
```

Табличное пространство по умолчанию — ts.

Вывод: если нет явного указания, табличное пространство по умолчанию определяется шаблоном, из которого клонируется новая база данных.

4. Символическая ссылка

```
=> SELECT oid AS tsoid FROM pg_tablespace WHERE spcname = 'ts';
```

```
 tsoid
-------
 16386
(1 row)
```

```
student$ sudo -u postgres ls -l /var/lib/postgresql/16/main/pg_tblspc/16386
```

```
lrwxrwxrwx 1 postgres postgres 26 ноя 25 13:48
```

```
/var/lib/postgresql/16/main/pg_tblspc/16386 -> /var/lib/postgresql/ts_dir
```

5. Удаление табличного пространства

```
=> ALTER DATABASE template1 SET TABLESPACE pg_default;
ALTER DATABASE
=> DROP DATABASE db;
DROP DATABASE
=> DROP TABLESPACE ts;
DROP TABLESPACE
student$ sudo -u postgres rm -rf /var/lib/postgresql/ts_dir
```

Практика+

1. Установите параметр *random_page_cost* в значение 1.1 для табличного пространства pg_default.

1. Используйте команду ALTER TABLESPACE ... SET:

https://postgrespro.ru/docs/postgresql/16/sql-altertablespace

Параметры *seq_page_cost* и *random_page_cost* используются планировщиком запросов и задают примерную стоимость чтения с диска одной страницы данных при последовательном и произвольном доступе соответственно.

Чем меньше соотношение между этими параметрами, тем чаще планировщик будет предпочитать индексный доступ последовательному сканированию таблицы.

Более подробно параметры **_cost,* и в частности *random_page_cost*, рассматриваются в курсе QPT «Оптимизация запросов».

1. Установка random_page_cost для табличного пространства

Значения по умолчанию параметров seq_page_cost и random_page_cost больше подходят для медленных HDD-дисков. Предполагается, что доступ к произвольной странице данных в четыре раза дороже последовательного:

```
=> \dconfig *page_cost
```

```
List of configuration parameters
    Parameter     | Value
------------------+-------
 random_page_cost | 4
 seq_page_cost    | 1
(2 rows)
```

Если используются диски с разными характеристиками, для них можно создать разные табличные пространства и настроить подходящие соотношения этих параметров. Например для быстрых SSD-дисков значение random_page_cost можно уменьшить практически до значения seq_page_cost.

```
=> ALTER TABLESPACE pg_default SET (random_page_cost = 1.1);
```

```
ALTER TABLESPACE
```

Настройки, сделанные командой ALTER TABLESPACE, сохраняются в таблице pg_tablespace. Их можно посмотреть в psql следующей командой:

```
=> \db+
```

```
                                          List of tablespaces
    Name    |  Owner   | Location | Access privileges |        Options         |  Size  |
Description
------------+----------+----------+-------------------+------------------------+--------+-
------------
 pg_default | postgres |          |                   | {random_page_cost=1.1} | 29 MB  |
 pg_global  | postgres |          |                   |                        | 589 kB |
(2 rows)
```

Параметры *_cost можно установить и в postgresql.conf. Тогда они будут действовать для всех табличных пространств.